PRODUCT REQUIREMENTS DOCUMENT (PRD)

DocuLens — AI-Powered Document Intelligence System

Document Type: Product Requirements Document + Engineering Implementation Specification **Product:** DocuLens **Version:** 1.1 **Priority:** MVP with Future-Ready Architecture **Implementation Deadline:** 4 Days **Product Owner:** AI Product Manager **Target Audience:** Technical Project Manager, AI Engineers, Backend Engineers, Frontend Engineers, QA Engineers **Document Status:** Implementation Baseline

1. Executive Summary

1.1 Product Overview

DocuLens is an AI-powered financial document intelligence platform designed to automate the ingestion, extraction, validation, reconciliation, and retrieval of financial information from multiple document formats.

Organizations receive large volumes of financial documents, including invoices, receipts, scanned PDFs, images, Excel spreadsheets, and CSV files.

Traditionally, employees manually read these documents, extract financial information, verify calculations, compare information across documents, and enter the results into internal systems.

This process is time-consuming, expensive, difficult to scale, and prone to human errors.

DocuLens aims to automate this workflow by combining:

- Document processing and OCR.
- LLM-powered structured information extraction.
- Deterministic financial validation.
- PDF-to-PDF reconciliation.
- PDF-to-Excel reconciliation.
- Human-in-the-Loop (HITL) review.
- Role-Based Access Control (RBAC).
- Centralized document storage and retrieval.
- Keyword, filtered, and semantic document search.
- REST API-based integration.

The system will provide a React-based web interface and a FastAPI backend.

1.2 Product Vision

Transform unstructured financial documents into structured, validated, traceable, and searchable financial information through intelligent automation.

1.3 Core Value Proposition

Reduce manual document-processing effort while improving financial data accuracy, processing speed, operational efficiency, and information accessibility.

1.4 Product Objectives

1. Automate financial information extraction from multiple document formats.
2. Reduce manual data entry and repetitive document-processing tasks.
3. Detect missing, incorrect, duplicated, and inconsistent financial information.
4. Compare financial information across related documents.
5. Provide human reviewers with an exception-focused review workflow.
6. Maintain centralized and searchable financial document records.
7. Provide secure access according to user roles.
8. Establish a modular architecture that supports future product expansion.

2. Business Context and Problem Statement

2.1 Existing Business Challenges

Organizations processing large volumes of financial documents face several challenges.

A. Manual document processing

Employees manually open documents, read financial information, extract relevant fields, and enter information into internal systems.

B. Financial inaccuracies

Manual data entry and calculations can introduce incorrect invoice amounts, taxes, dates, and other financial information.

C. Slow processing

Processing thousands of documents manually creates operational bottlenecks.

D. Document reconciliation difficulties

Employees manually compare invoices with other PDFs or financial spreadsheets to identify inconsistencies.

E. Poor information accessibility

Historical financial documents and extracted information are often scattered across multiple locations.

F. Limited scalability

Increasing document volume requires additional human resources when the processing workflow is predominantly manual.

2.2 Expected Business Transformation

Current workflow:

Document collection → Manual reading → Manual extraction → Manual calculation → Manual reconciliation → Manual data entry.

Target workflow:

Document upload → Automated document understanding → AI extraction → Deterministic validation → Cross-document reconciliation → Exception-based human review → Structured data storage → Search and retrieval.

3. Target Users and Role Definitions

DocuLens will initially support four user roles.

The implementation must include functional RBAC rather than a single unrestricted administrator account.

3.1 Finance Executive

Primary responsibility: Upload and process financial documents.

Permissions:

- Upload multiple documents.
- View documents uploaded by authorized users.
- Monitor processing status.
- View extracted financial information.
- View validation results.
- Perform permitted manual corrections.
- Initiate document reconciliation.
- Search accessible documents.
- View processing history.

3.2 Finance Manager

Primary responsibility: Review financial discrepancies and oversee processing quality.

Permissions:

- All Finance Executive permissions.
- View all documents within the organization.
- Review flagged documents.
- Approve or reject reviewed records.
- Approve manual corrections.
- Review reconciliation results.
- View audit history.
- Monitor processing and review statistics.

3.3 Accounts / Audit Team

Primary responsibility: Investigate financial inconsistencies and support financial verification.

Permissions:

- View authorized documents.
- Search financial records.
- Inspect extracted financial information.
- View validation errors.
- Initiate PDF-to-PDF reconciliation.
- Initiate PDF-to-Excel reconciliation.
- View reconciliation results.
- Export or download permitted reports.

Default restriction:

- Cannot modify extracted financial information.
- Cannot approve financial records.
- Cannot manage users.

3.4 System Administrator

Primary responsibility: Manage users, permissions, and system operations.

Permissions:

- Create, update, and deactivate users.
- Assign user roles.
- View all documents.
- Monitor processing jobs.
- Retry failed processing jobs.
- View system-level processing history.
- Manage system configuration.
- Access audit logs.

3.5 Permission Matrix

Capability	Finance Executive	Finance Manager	Accounts/Audit	Admin
Upload documents	Yes	Yes	Yes	Yes
View documents	Authorized	All	Authorized	All
View extraction	Yes	Yes	Yes	Yes
View validation	Yes	Yes	Yes	Yes
Edit extracted data	Yes, permitted records	Yes	No	Administrative
Approve corrections	No	Yes	No	Administrative
Reconciliation	Yes	Yes	Yes	Yes

Search documents	Yes	Yes	Yes	Yes
Manage users	No	No	No	Yes
Retry failed jobs	No	No	No	Yes
View audit history	Own/authorized	All	Authorized	All

Implementation note: Enforce authorization on the backend. Hiding frontend buttons is not an access-control mechanism.

For the MVP, use a single organization with four roles. Design the authorization layer so additional roles and organization-level isolation can be introduced later.

4. Product Scope

4.1 MVP Scope — Must Implement

The following capabilities are mandatory for the initial release.

ID	Feature	Priority
F-01	Authentication and four-role RBAC	P0
F-02	Multi-format document upload	P0
F-03	Digital PDF text extraction	P0
F-04	OCR for scanned PDFs and images	P0
F-05	Excel and CSV parsing	P0
F-06	LLM-based invoice information extraction	P0

F-07	Structured JSON output	P0
F-08	Deterministic financial validation	P0
F-09	Duplicate invoice detection	P0
F-10	Exception-based human review	P0
F-11	Manual correction and approval	P0
F-12	PDF-to-PDF reconciliation	P0
F-13	PDF-to-Excel reconciliation	P0
F-14	Background processing and job tracking	P0
F-15	PostgreSQL data persistence (with pgvector)	P0
F-16	Keyword search and structured filtering	P0
F-17	Processing history and audit trail	P0
F-18	Failed-job reprocessing	P0
F-19	REST API	P0
F-20	Automated tests and evaluation	P0
F-21	Semantic and hybrid document search (pgvector)	P0

4.2 Deferred Capabilities

The following capabilities are intentionally excluded from the initial implementation.

However, the architecture must not prevent their future introduction.

Feature	Current implementation	Future direction

Natural-language query retrieval	Keyword + semantic search with structured filters	LLM-parsed natural-language queries, reranking, conversational retrieval
Advanced PDF reconciliation	Simple field-level comparison	Intelligent document matching and reconciliation
Advanced PDF-to-Excel reconciliation	Explicit field mapping and comparison	Automated column mapping and complex financial reconciliation
Enterprise RBAC	Four predefined roles	Custom roles, granular permissions, organization-level policies
Large-scale distributed processing	Database-backed job queue with worker processes	Dedicated message broker (Redis with Celery or equivalent), horizontal worker scaling and distributed processing
Advanced analytics dashboards	Basic processing and review statistics	Operational analytics, financial insights, trend analysis
Multi-tenancy	Single organization	Organization-level data isolation

4.3 Architectural Principle

Deferred features must not require major changes to the existing core architecture.

This does not mean implementing their entire infrastructure now.

It means:

- Use modular service boundaries.
- Avoid tightly coupling business logic with API routes.
- Separate document extraction from financial validation.
- Separate reconciliation logic from document extraction.
- Use configurable validation rules.
- Keep database relationships extensible.

- Maintain structured processing and audit history.
- Use a role-permission authorization abstraction.
- Design background jobs independently of individual processing tasks.

Do not build unnecessary abstractions or infrastructure solely for hypothetical future requirements.

5. Functional Requirements

5.1 Authentication and Authorization

Feature ID: F-01

Objective

Provide secure authentication and role-based access control for four predefined user roles.

Functional requirements

FR-001: Users must authenticate before accessing protected application features.

FR-002: The system must support Finance Executive, Finance Manager, Accounts/Audit, and System Administrator roles.

FR-003: Each user must have an assigned role.

FR-004: The backend must validate user permissions for protected operations.

FR-005: Unauthorized operations must return HTTP 403.

FR-006: Unauthenticated requests must return HTTP 401.

FR-007: Administrators must be able to create, update, deactivate, and assign roles to users.

FR-008: User passwords must be securely hashed.

FR-009: Sensitive authentication credentials must not be stored in frontend source code.

FR-010: User identity and role must be available to the backend during request processing.

Future extensibility

Implement an authorization service or dependency-based permission layer.

Avoid hardcoding role checks independently inside every endpoint.

A future permission model should support:

- Permission identifiers.
- Role-permission mappings.
- Custom roles.
- Organization-specific permissions.

For the MVP, a predefined role-permission mapping is sufficient.

5.2 Document Upload and Ingestion

Feature ID: F-02

Objective

Allow users to upload multiple financial documents through the web application and REST API.

Supported formats

- PDF
- JPG
- JPEG
- PNG
- TIFF
- XLSX
- XLS

- CSV

Functional requirements

FR-011: Users must be able to upload multiple files in one operation.

FR-012: The system must validate file extensions and actual file types.

FR-013: The system must reject unsupported file formats.

FR-014: The system must enforce configurable file-size limits.

FR-015: The system must generate a unique document ID for every uploaded file.

FR-016: The system must store original files separately from extracted and processed data.

FR-017: The system must calculate a file checksum to support duplicate-file detection.

FR-018: The system must record document metadata, including:

- Original filename.
- File type.
- File size.
- Checksum.
- Uploaded by.
- Upload timestamp.
- Processing status.

FR-019: Uploading a file must create a processing job.

FR-020: Users must receive a processing identifier or document ID to track the operation.

Error handling

The system must handle:

- Unsupported file types.
- Empty files.
- Corrupted files.
- Duplicate uploads.

- Oversized files.
- Upload interruptions.

Future extensibility

Use a storage abstraction rather than coupling document processing directly to local filesystem paths.

For the MVP, local storage is acceptable.

Future storage implementations may include S3-compatible object storage or other cloud storage providers.

5.3 Document Processing and Text Extraction

Feature ID: F-03, F-04, F-05

Objective

Convert supported documents into machine-readable content.

A. Digital PDF extraction

Use PyMuPDF or an equivalent Python PDF processing library.

Requirements:

FR-021: Extract text from digital PDFs.

FR-022: Support multi-page documents.

FR-023: Preserve page-level text and page numbers.

FR-024: Identify pages containing insufficient or missing extractable text.

FR-025: Route scanned or image-based pages to OCR.

B. OCR processing

Use Tesseract or PaddleOCR.

Requirements:

FR-026: Extract text from JPG, JPEG, PNG, and TIFF files.

FR-027: Extract text from scanned PDF pages.

FR-028: Support multi-page scanned PDFs.

FR-029: Preserve page-level extraction results.

FR-030: Handle poor-quality images as gracefully as possible.

FR-031: Record OCR failures and extraction warnings.

FR-032: Never silently treat an OCR failure as an empty successful extraction.

C. Spreadsheet processing

Use pandas and openpyxl.

Requirements:

FR-033: Support XLSX, XLS, and CSV.

FR-034: Read multiple sheets from Excel workbooks.

FR-035: Preserve sheet names.

FR-036: Extract tabular information, including headers and rows.

FR-037: Handle missing values and inconsistent column structures.

FR-038: Preserve enough source information to trace extracted spreadsheet values to their original sheet and row.

D. Unified extraction output

All document processors must produce a common internal representation.

Suggested structure:

```
{
  "document_id": "uuid",
  "document_type": "PDF",
  "extraction_method": "DIGITAL_TEXT",
  "pages": [
    {
      "page_number": 1,
      "text": "Extracted page content"
    }
  ],
  "warnings": [],
  "processing_status": "COMPLETED"
}
```

For spreadsheets, use a corresponding structure containing sheets, headers, and rows.

Architectural requirement

Implement format-specific processors behind a common document-processing interface.

Example:

```
class DocumentProcessor:
    def can_process(self, file_type: str) -> bool:
        raise NotImplementedError

    def process(self, file_path: str):
        raise NotImplementedError
```

Implement separate processors:

- PDFProcessor
- ImageOCRProcessor
- ExcelProcessor
- CSVProcessor

The processing orchestrator should select the appropriate processor.

Adding a future file format must not require rewriting the existing processing pipeline.

5.4 AI-Based Financial Information Extraction

Feature ID: F-06, F-07

Objective

Extract structured invoice information from processed document content using an LLM.

Required invoice fields

The system must extract:

1. Invoice number.
2. Invoice date.
3. Supplier name.
4. Customer name.
5. Subtotal.
6. Tax.
7. Total amount.
8. Currency.
9. Line items.

Line-item structure

Each line item should support:

- Description.
- Quantity.
- Unit price.
- Amount.

Optional fields such as tax rate or discount may be supported when present, but are not mandatory for the MVP.

Functional requirements

FR-039: Extract all required fields from the available document content.

FR-040: Return structured JSON.

FR-041: Do not invent missing information.

FR-042: Return null for unavailable or unidentifiable fields.

FR-043: Preserve original source values where possible.

FR-044: Identify uncertain extraction results.

FR-045: Provide field-level confidence or an equivalent uncertainty indicator.

FR-046: Validate the LLM response against a predefined schema.

FR-047: Retry or handle malformed LLM responses safely.

FR-048: Preserve the original extraction output for auditability.

Example output

```
{
  "invoice_number": "INV-2026-001",
  "invoice_date": "2026-10-25",
  "supplier_name": "ABC Technologies",
  "customer_name": "XYZ Corporation",
  "subtotal": 1000.00,
  "tax": 100.00,
  "total": 1100.00,
  "currency": "USD",
  "line_items": [
    {
      "description": "Software License",
      "quantity": 2,
      "unit_price": 500.00,
```

```
      "amount": 1000.00
    }
  ],
  "field_confidence": {
    "invoice_number": 0.98,
    "invoice_date": 0.95,
    "supplier_name": 0.97,
    "total": 0.68
  }
}
```

The confidence values above are illustrative.

Do not assume that LLM-reported confidence represents statistically calibrated accuracy.

Extraction architecture

Separate the following responsibilities:

1. Document text extraction.
2. Prompt construction.
3. LLM invocation.
4. JSON parsing.
5. Schema validation.
6. Field normalization.
7. Confidence and uncertainty handling.
8. Persistence of extracted results.

Do not place all these operations inside one large API route or one tightly coupled function.

Important processing consideration

A PDF can contain hundreds or thousands of pages.

Do not send an entire large PDF's extracted text directly into a single LLM request.

For the MVP:

- Process documents page by page or in manageable page batches.

- Identify relevant invoice content.
- Use controlled context windows.
- Aggregate extraction results when a document spans multiple pages.
- Preserve page references for extracted fields where feasible.

The extraction strategy should be designed so more advanced document chunking and retrieval can be introduced later.

5.5 Financial Data Validation

Feature ID: F-08

Objective

Verify extracted financial information using deterministic backend rules.

LLMs must not be responsible for final financial validation.

Validation categories

A. Required-field validation

Check the presence of:

- Invoice number.
- Invoice date.
- Supplier name.
- Total amount.
- Currency, when required by the business rules.

Missing fields should generate validation issues.

B. Numeric validation

Check:

- Numeric values are valid.

- Amounts are not malformed.
- Quantities are valid.
- Negative values are handled according to configurable business rules.

Do not universally reject negative amounts because credit notes and adjustments may legitimately contain them.

C. Financial calculation validation

Check:

```
Expected Total = Subtotal + Tax
```

Where discounts, shipping, or other charges are available:

```
Expected Total =
Subtotal + Tax + Additional Charges - Discounts
```

For line items:

```
Line Item Amount = Quantity × Unit Price
 Expected Subtotal = Sum of Line Item Amounts
```

Apply configurable rounding tolerances.

Currency precision must be considered when performing financial comparisons.

D. Duplicate detection

Identify possible duplicate invoices using:

- Supplier name or normalized supplier identifier.
- Invoice number.
- Invoice date.
- Total amount.

Use exact matching as the primary duplicate-detection mechanism.

Fuzzy matching can be introduced later.

A duplicate candidate should be flagged for review, not automatically deleted or rejected.

E. Validation output

Suggested structure:

```
{
  "is_valid": false,
  "validation_status": "NEEDS_REVIEW",
  "errors": [
    {
      "code": "TOTAL_MISMATCH",
      "field": "total",
      "message": "Subtotal plus tax does not match invoice total.",
      "expected_value": 1100.00,
      "actual_value": 1010.00,
      "severity": "ERROR"
    }
  ],
  "warnings": []
}
```

Architectural requirement

Implement validation rules independently from extraction logic.

Suggested components:

- RequiredFieldValidator.
- NumericValidator.
- FinancialCalculationValidator.
- DuplicateInvoiceValidator.

Use a centralized validation orchestrator.

Future validation rules must be addable without modifying the extraction engine.

5.6 Human-in-the-Loop Review

Feature ID: F-10, F-11

Objective

Ensure that uncertain or inconsistent financial records receive human attention before being treated as verified business records.

Review strategy

Implement exception-based human review.

Not every invoice requires manual approval.

Invoices that pass configured automated checks may proceed automatically. Invoices with detected issues must enter the review workflow.

Review triggers

A document should require review when:

- Required information is missing.
- Critical fields have insufficient extraction confidence.
- Financial calculations are inconsistent.
- Duplicate invoices are suspected.
- Cross-document reconciliation detects discrepancies.
- OCR or extraction warnings require human investigation.

Processing and review statuses

Keep processing status and business review status separate.

Processing status:

- PENDING.
- PROCESSING.

- COMPLETED.
- FAILED.

Review status:

- NOT_REQUIRED.
- PENDING.
- IN_REVIEW.
- CORRECTED.
- APPROVED.
- REJECTED.

A document may have:

```
processing_status = COMPLETED
review_status = PENDING
```

This means extraction has finished, but human review is required.

Review workflow

1. User opens the review queue.
2. User selects a flagged document.
3. System displays the original document.
4. System displays extracted financial fields.
5. System highlights validation errors.
6. Authorized user corrects inaccurate values.
7. Corrected values are saved.
8. Finance Manager reviews and approves the record.
9. The system preserves the original extraction and correction history.

Audit requirements

Record:

- Original extracted value.
- Corrected value.
- User who made the correction.

- Timestamp.
- Correction reason, when provided.
- Approval status.
- Approving user.

Never overwrite the original extraction without maintaining its history.

Automatic approval policy

Documents without detected issues may receive:

```
review_status = NOT_REQUIRED
```

This means no operational review is required.

It does not guarantee that the document contains no undetected errors.

A future quality-audit mechanism may sample automatically approved documents.

5.7 PDF-to-PDF Reconciliation

Feature ID: F-12

Objective

Compare financial information extracted from two related PDF documents and identify discrepancies.

MVP functionality

Users must be able to select two processed PDF documents and initiate reconciliation.

Example:

Document A: Supplier Invoice.pdf

Document B: Purchase Statement.pdf

The system compares available financial fields.

Fields to compare

- Invoice number.
- Invoice date.
- Supplier name.
- Customer name.
- Subtotal.
- Tax.
- Total.
- Currency.

Compare line items where both documents contain sufficiently structured line-item information.

Reconciliation workflow

1. User selects the primary PDF.
2. User selects the comparison PDF.
3. System verifies that both documents have completed extraction.
4. System loads their structured results.
5. System compares corresponding fields.
6. System generates a reconciliation report.
7. User reviews mismatched fields.

Matching rules

For the MVP:

- Invoice number: normalized exact comparison.
- Supplier/customer: normalized text comparison.
- Invoice date: normalized date comparison.
- Currency: exact normalized comparison.
- Financial amounts: configurable numeric tolerance.
- Line items: compare descriptions and amounts where reliable matching is possible.

If a field is unavailable in one document, report it as NOT_COMPARABLE rather than inventing a mismatch.

Example reconciliation result

```
{
  "reconciliation_id": "uuid",
  "source_document_id": "uuid-1",
  "target_document_id": "uuid-2",
  "status": "MISMATCH_FOUND",
  "results": [
    {
      "field": "invoice_number",
      "source_value": "INV-001",
      "target_value": "INV-001",
      "result": "MATCH"
    },
    {
      "field": "total",
      "source_value": 1100.00,
      "target_value": 1010.00,
      "result": "MISMATCH"
    }
  ]
}
```

Reconciliation statuses

- MATCHED.
- MISMATCH_FOUND.
- PARTIALLY_MATCHED.
- NOT_COMPARABLE.
- FAILED.

Future extensibility

Reconciliation must be implemented as an independent service.

Future versions should support:

- More than two documents.
- Automatic document relationship detection.
- Different document types.
- Configurable matching strategies.
- Fuzzy matching.
- Complex financial reconciliation.
- Batch reconciliation.

Do not couple reconciliation directly to the PDF extraction processor.

5.8 PDF-to-Excel Reconciliation

Feature ID: F-13

Objective

Compare extracted financial information from a PDF invoice against corresponding records in an Excel workbook.

MVP functionality

Users must be able to select:

- One processed PDF invoice.
- One processed Excel workbook.
- One sheet containing the relevant financial records.

The user must be able to identify or map the required Excel columns.

Example

PDF invoice:

Field	Value

Invoice Number	INV-001
Supplier	ABC Technologies
Subtotal	1,000
Tax	100
Total	1,100

Excel record:

Invoice Number	Supplier	Subtotal	Tax	Total
INV-001	ABC Technologies	1,000	100	1,010

Expected result: Total mismatch.

Functional requirements

FR-049: Users must be able to select an existing processed PDF.

FR-050: Users must be able to select an existing processed Excel workbook.

FR-051: Users must be able to select the target worksheet.

FR-052: The system must display available spreadsheet column headers.

FR-053: Users must be able to map invoice fields to Excel columns.

FR-054: The system must identify the corresponding Excel record using an invoice identifier, where available.

FR-055: The system must compare mapped financial fields.

FR-056: The system must generate a field-level reconciliation report.

FR-057: Missing or ambiguous records must be reported clearly.

Suggested default mapping

PDF field	Excel column
Invoice number	Invoice Number
Supplier	Supplier
Invoice date	Invoice Date
Subtotal	Subtotal
Tax	Tax
Total	Total
Currency	Currency

Column names must not be assumed to be identical.

Allow explicit mapping because different companies may use different spreadsheet structures.

Future extensibility

Keep spreadsheet parsing separate from reconciliation logic.

Implement a mapping layer that can later support:

- Saved mapping templates.
- Automatic column detection.
- Multiple matching records.
- Batch reconciliation.
- Additional spreadsheet formats.

5.9 Background Processing and Job Management

Feature ID: F-14, F-18

Objective

Process time-consuming document operations asynchronously without blocking API requests.

Proposed implementation

Use:

- A database-backed job queue built on the `processing_jobs` table in PostgreSQL.
- One or more Python worker processes started separately from the API (`python -m app.worker`).

No external message broker is used in the MVP. PostgreSQL is the single source of truth for job state.

The unit of work is one function, `run_job(job_id)`, independent of how it is triggered. The worker loop is a thin driver around it, so a message broker can replace polling later without changing pipeline code.

Functional requirements

FR-058: Upload requests must not wait for OCR, LLM, and embedding processing to finish.

FR-059: The backend must create processing jobs.

FR-060: Workers must process documents asynchronously.

FR-061: Processing status must be persisted.

FR-062: Users must be able to retrieve job status.

FR-063: Failed jobs must record meaningful error information.

FR-064: Authorized administrators must be able to reprocess failed jobs.

FR-065: Reprocessing must not create duplicate document records.

FR-066: Workers must handle retryable failures using bounded retries with backoff.

FR-067: Jobs must be idempotent wherever practical.

Job types

The MVP defines two job types:

- DOCUMENT_PROCESSING: parsing, OCR, extraction, validation.
- EMBEDDING: page-text chunking and embedding generation (see section 5.11).

The EMBEDDING job is created after document processing completes. Its failure must not change the document's processing status.

Job claiming and leasing

- Workers claim the oldest runnable job (status PENDING and run_after not in the future) using `SELECT ... FOR UPDATE SKIP LOCKED`, set status to PROCESSING, set `locked_at`, increment `attempt_count`, and commit the claim before processing begins.
- Multiple workers can run concurrently without claiming the same job.
- Workers update `locked_at` after each pipeline stage.
- A PROCESSING job whose `locked_at` is older than a configurable timeout is treated as abandoned and returned to PENDING. This counts as an attempt.
- At most one active (PENDING or PROCESSING) job may exist per document and job type.
- The worker poll interval, maximum attempts, backoff, and stuck-job timeout are configurable.

Job lifecycle

```
PENDING
   |
   v
PROCESSING
   |
```

```
+------> COMPLETED
|
+------> PENDING   (retryable failure, attempts remaining,
|                   run_after = now + backoff)
|
+------> FAILED    (non-retryable failure or attempts exhausted)
```

An administrator can manually retry a FAILED job, which sets it to PENDING and resets `run_after`.

Important architectural requirement

Do not execute OCR, LLM calls, embedding calls, or spreadsheet processing directly inside HTTP request handlers.

Separate API orchestration from worker execution.

Use the database as the source of truth for processing status.

Pipeline stages must be idempotent. Each stage replaces its own output for the document inside a transaction, so retries and reprocessing cannot create duplicate rows.

Future extensibility

The worker architecture should support adding new task types without modifying the upload API.

A dedicated message broker (Redis with Celery or equivalent) can replace database polling when throughput or scheduling needs justify it.

Potential future tasks:

- Document classification.
- Advanced reconciliation.
- Batch processing.
- Analytics aggregation.
- Re-embedding after an embedding-model change.

5.10 Database and Data Management

Feature ID: F-15

Objective

Maintain a structured and auditable database for documents, extraction results, validation issues, reconciliation results, users, processing history, and searchable document content.

Proposed database

PostgreSQL with the pgvector and pg_trgm extensions.

Hosting is a deployment detail (hosted PostgreSQL such as Supabase or Neon, or a local installation with the extensions) and must be configurable through `DATABASE_URL`.

Recommended entities

1. users

Stores application users.

Fields:

- id
- name
- email
- password_hash
- role_id
- is_active
- created_at
- updated_at

2. roles

Stores predefined roles.

Fields:

- id
- name
- description

3. documents

Stores uploaded document metadata.

Fields:

- id
- original_filename
- storage_path
- mime_type
- file_size
- checksum
- document_type
- processing_status
- review_status
- uploaded_by
- created_at
- updated_at

4. document_pages

Stores page-level extraction results.

Fields:

- id
- document_id
- page_number
- extracted_text

- extraction_method
- extraction_metadata

5. spreadsheet_sheets

Stores workbook sheet information.

Fields:

- id
- document_id
- sheet_name
- sheet_index
- headers
- structured_data

6. extracted_invoices

Stores structured financial information.

The structured columns hold the current values, including approved manual corrections. The original LLM output is preserved in `extraction_metadata` and in `document_corrections`.

Fields:

- id
- document_id
- invoice_number
- invoice_date
- supplier_name
- customer_name
- subtotal
- tax
- total
- currency
- extraction_metadata
- created_at

- updated_at

7. invoice_line_items

Stores invoice line items.

Fields:

- id
- invoice_id
- description
- quantity
- unit_price
- amount

8. validation_results

Stores validation issues.

Fields:

- id
- document_id
- validation_code
- field_name
- severity
- message
- expected_value
- actual_value
- status
- created_at

9. reconciliation_jobs

Stores reconciliation operations.

Fields:

- id

- reconciliation_type
- source_document_id
- target_document_id
- status
- initiated_by
- created_at
- completed_at

10. reconciliation_results

Stores field-level comparison results.

Fields:

- id
- reconciliation_job_id
- field_name
- source_value
- target_value
- comparison_status
- metadata

11. document_corrections

Stores manual correction history.

Fields:

- id
- document_id
- invoice_id
- field_name
- original_value
- corrected_value
- corrected_by
- correction_reason
- created_at

12. processing_jobs

Stores background processing information.

Fields:

- - id
- - document_id
- - job_type (DOCUMENT_PROCESSING, EMBEDDING)
- - status
- - attempt_count
- - run_after
- - locked_at
- - error_message
- - started_at
- - completed_at
- - created_at

13. audit_logs

Stores important user and system activities.

Fields:

- id
- user_id
- action
- entity_type
- entity_id
- old_values
- new_values
- created_at

14. document_chunks

Stores searchable text segments and their embeddings. Chunks are derived from `document_pages.extracted_text`.

Fields:

- id
- document_id
- page_id
- chunk_index
- content
- content_tsv (generated tsvector column for keyword search)
- embedding (pgvector column; dimension defined by the configured embedding model)
- embedding_model
- created_at

Database design principles

1. Use UUID primary keys.
2. Maintain foreign-key relationships.
3. Use indexes for frequently queried fields.
4. Use numeric/decimal types for financial amounts, never floating-point types.
5. Use JSONB for flexible extraction metadata and structured source information where appropriate.
6. Preserve original extraction results.
7. Keep reconciliation history independent of document records.
8. Keep validation results independent of extraction logic.
9. Use database migrations.
10. Avoid storing original binary documents directly inside PostgreSQL for the MVP.
11. Enable pgvector in PostgreSQL. Do not introduce a separate vector database.
12. Record `embedding_model` on every chunk. Changing the embedding model changes the vector dimension and requires re-embedding.
13. Index design: an approximate nearest-neighbor index (HNSW, cosine distance) on `document_chunks.embedding`, a GIN index on `content_tsv`, and B-tree indexes on frequently filtered fields (invoice number, supplier name, invoice date, total, processing status, review status, uploaded_by).
14. Chunks and their embeddings for a document are written in one transaction, so a failed embedding job leaves no partial index for that document.
15. Use direct or session-mode database connections for migrations and workers. Transaction-pooled connections can break prepared statements.

Future extensibility

Design the schema so future capabilities can introduce:

- Organization-level ownership.
- Custom roles and permissions.
- Multiple reconciliation sources.
- Advanced document relationships.
- Analytics aggregates.

Do not create unused tables and services for every future possibility.

5.11 Document Search

Feature ID: F-16, F-21

Objective

Allow users to retrieve previously processed documents and their extracted financial information using keyword search, structured filters, and semantic search.

Search modes

Mode	Behavior
`keyword`	Exact and full-text matching on structured invoice fields, filename, and extracted page text
`semantic`	Vector similarity between the query embedding and document chunk embeddings
`hybrid`	Keyword and semantic ranked lists combined with Reciprocal Rank Fusion (RRF)

All modes accept the same structured filters. Filters are applied in SQL before ranking.

The default mode is configurable. It is set from the retrieval evaluation results (section 10), not assumed.

Searchable fields

- Invoice number.
- Supplier name.
- Customer name.
- Document filename.
- Invoice date.
- Total amount.
- Currency.
- Extracted page text (chunks).

Filters

- Processing status.
- Review status.
- Upload date range.
- Supplier.
- Invoice date range.
- Total amount range.
- Reconciliation status.

Functional requirements

FR-068: Users must be able to search by keyword across invoice number, supplier name, customer name, document filename, and extracted page text.

FR-069: Users must be able to apply the structured filters listed above, alone or combined with a query.

FR-070: Users must be able to run semantic search over document chunk embeddings.

FR-071: The system must support a hybrid mode that combines keyword and semantic rankings.

FR-072: Filters must be applied before ranking in every mode.

FR-073: Search results must be restricted to documents the requesting user is authorized to view, using the same visibility rule as the document list. Authorization must be applied before ranking so vector search cannot return unauthorized documents.

FR-074: Results must be returned one per document, including the best-matching chunk snippet and its page number where applicable.

FR-075: Results must be paginated and include ranking information (keyword rank, semantic similarity, and combined score) to support debugging.

FR-076: Semantic results must respect a configurable minimum similarity threshold. The threshold must be set from evaluation results, not guessed.

FR-077: An embedding failure must not fail document processing. Documents without chunks remain searchable by keyword and filters. If the query embedding fails, `semantic` mode returns a clear error and `hybrid` mode falls back to keyword results with a warning in the response.

FR-078: Semantic and hybrid search must only compare against chunks created with the currently configured embedding model.

FR-079: Structured-field search and filters must use current (corrected) values from `extracted_invoices`.

FR-080: Spreadsheet documents are searchable by filename and metadata only. Spreadsheet rows are not embedded in the MVP.

Embedding pipeline

- After document processing completes, an EMBEDDING job (section 5.9) splits `document_pages.extracted_text` into chunks and generates embeddings.
- Chunk size and overlap are configurable.
- The embedding provider and model are configurable and sit behind a provider interface. One model is pinned per deployment.
- The embedding job is retryable through the existing job retry mechanism.
- Corrections to structured invoice fields do not require re-embedding because chunks are derived from extracted page text.
- Changing the embedding model requires a re-embedding run (administrative script or job), not a code change.

Relevance ranking

- Keyword rank: PostgreSQL full-text rank on chunk text, with exact matches on invoice number and supplier name ranked first.
- Semantic rank: cosine similarity on chunk embeddings.
- Hybrid: RRF over the two ranked lists, aggregated to one result per document using the best chunk.

Example queries

- Find invoice INV-001. *(keyword)*
- Find all invoices from ABC Technologies. *(keyword / filter)*
- Find invoices with totals greater than $1,000. *(filter)*
- Find documents requiring manual review. *(filter)*
- Find failed processing jobs. *(filter)*
- Find documents mentioning late delivery penalties. *(semantic)*

Architectural requirements

Search functionality must be implemented behind a search service or repository abstraction.

Embedding generation must be implemented behind a provider interface so the provider can be replaced without changing search or pipeline logic.

Deferred search capabilities

Not part of the MVP:

- Natural-language query parsing into structured filters.
- Conversational or multi-turn retrieval.
- Reranker models.
- Fine-tuned embeddings.
- A separate vector database.
- Row-level spreadsheet indexing.

5.12 Frontend Application

Feature ID: Frontend

Technology

React + TypeScript.

Objective

Provide a straightforward interface for document upload, monitoring, extraction review, reconciliation, and search.

Required application modules

A. Authentication

- Login page.
- Authenticated session management.
- Role-aware navigation.

B. Dashboard

Display basic operational statistics:

- Total documents.
- Processing documents.
- Completed documents.
- Failed documents.
- Documents requiring review.
- Reconciliation operations.

These are basic operational indicators, not an advanced analytics dashboard.

C. Document Upload

- Multiple file selection.
- Drag-and-drop upload.

- File type validation.
- Upload progress.
- Upload success/failure feedback.

D. Document List

Display:

- Filename.
- File type.
- Uploaded by.
- Upload timestamp.
- Processing status.
- Review status.
- Available actions.

Support pagination and filtering.

E. Document Details

Display:

- Original document preview or download.
- Extracted text.
- Extracted financial information.
- Extraction warnings.
- Validation errors.
- Processing history.

F. Review Queue

Display documents requiring human review.

Users should be able to:

- Open a document.
- Inspect extracted values.
- View highlighted validation issues.
- Correct permitted fields.

- Submit corrections.
- Approve or reject records according to their role.

G. Reconciliation Module

Provide two operations:

1. PDF-to-PDF.
2. PDF-to-Excel.

Display field-level matches, mismatches, missing values, and overall reconciliation status.

H. Search Module

Provide:

- A search box with a mode selector (keyword, semantic, hybrid).
- Structured filters.
- Paginated results showing the matched snippet and page reference.
- A warning indicator when hybrid search falls back to keyword results.

I. User Management

Accessible only to the System Administrator.

Support:

- User listing.
- User creation.
- User updates.
- Role assignment.
- User deactivation.

Frontend architectural requirements

Use:

- Feature-based component organization.
- Centralized API client.

- Reusable form components.
- Shared TypeScript types.
- Role-aware route protection.
- Consistent loading, error, and empty states.

Avoid placing business logic inside UI components.

The frontend must consume backend APIs rather than implementing validation or reconciliation independently.

6. API Requirements

The backend must expose REST APIs through FastAPI.

All API responses should follow a consistent response format.

Example:

```
{
  "success": true,
  "message": "Operation completed successfully",
  "data": {},
  "error": null
}
```

Use appropriate HTTP status codes and structured error responses.

6.1 Authentication APIs

Method	Endpoint	Purpose
POST	/api/v1/auth/login	Authenticate user
POST	/api/v1/auth/refresh	Refresh access token

GET	/api/v1/auth/me	Retrieve current user

6.2 User Management APIs

Method	Endpoint	Purpose
GET	/api/v1/users	List users
POST	/api/v1/users	Create user
GET	/api/v1/users/{id}	Get user details
PATCH	/api/v1/users/{id}	Update user
PATCH	/api/v1/users/{id}/role	Assign role
PATCH	/api/v1/users/{id}/status	Activate/deactivate user

6.3 Document APIs

Method	Endpoint	Purpose
POST	/api/v1/documents/upload	Upload multiple documents
GET	/api/v1/documents	List documents
GET	/api/v1/documents/{id}	Document details
GET	/api/v1/documents/{id}/file	Original document access
GET	/api/v1/documents/{id}/status	Processing status
GET	/api/v1/documents/{id}/extraction	Extracted information
GET	/api/v1/documents/{id}/validation	Validation results

6.4 Review APIs

Method	Endpoint	Purpose
GET	/api/v1/reviews	List documents requiring review
GET	/api/v1/reviews/{document_id}	Review details
PATCH	/api/v1/reviews/{document_id}/corrections	Submit corrections
POST	/api/v1/reviews/{document_id}/approve	Approve record
POST	/api/v1/reviews/{document_id}/reject	Reject record

6.5 Reconciliation APIs

Method	Endpoint	Purpose
POST	/api/v1/reconciliation/pdf-pdf	Initiate PDF comparison
POST	/api/v1/reconciliation/pdf-excel	Initiate PDF-Excel comparison
GET	/api/v1/reconciliation	List reconciliation jobs
GET	/api/v1/reconciliation/{id}	Get reconciliation results

Example PDF-to-PDF request:

```
{
  "source_document_id": "uuid-1",
  "target_document_id": "uuid-2"
}
```

Example PDF-to-Excel request:

```
{
  "pdf_document_id": "uuid-1",
  "excel_document_id": "uuid-2",
```

```
  "sheet_name": "Invoices",
  "column_mapping": {
    "invoice_number": "Invoice Number",
    "supplier_name": "Supplier",
    "subtotal": "Subtotal",
    "tax": "Tax",
    "total": "Total"
  }
}
```

6.6 Search APIs

Method	Endpoint	Purpose
GET	/api/v1/search/documents	Keyword, semantic, and hybrid search with filters

Query parameters:

- q: search text (optional when only filters are used).
- mode: keyword, semantic, or hybrid.
- Filters: processing_status, review_status, supplier, upload_from, upload_to, invoice_date_from, invoice_date_to, min_total, max_total, reconciliation_status.
- page, page_size.

Each result includes the document summary, matched snippet, page number, and ranking information (keyword rank, semantic similarity, combined score). The response includes a warning field when hybrid search falls back to keyword results.

Example:

```
GET
/api/v1/search/documents?q=late%20delivery%20penalty&mode=hybrid&revie
w_status=PENDING&page=1&page_size=20
```

6.7 Processing APIs

Method	Endpoint	Purpose
GET	/api/v1/jobs	List processing jobs
GET	/api/v1/jobs/{id}	Job details
POST	/api/v1/jobs/{id}/retry	Retry failed job

All protected endpoints must enforce the appropriate role permissions.

7. Proposed Technical Architecture

7.1 Technology Stack

Layer	Technology
Frontend	React + TypeScript
UI styling	Tailwind CSS + shadcn/ui
Backend	Python + FastAPI
Database	PostgreSQL + pgvector + pg_trgm (hosted: Supabase or Neon)
ORM	SQLAlchemy
Database migrations	Alembic
Background Jobs	PostgreSQL-backed queue (`processing_jobs`) + Python worker process
Digital PDF extraction	PyMuPDF

OCR	Tesseract or PaddleOCR
Spreadsheet processing	pandas + openpyxl
AI extraction	LLM API with structured output
Embeddings	Provider selected during implementation, behind a provider interface
Authentication	JWT-based authentication
Search	PostgreSQL full-text search + pgvector similarity; no separate vector database
File type validation	Magic-byte signature checks (no native dependencies)
Local development	Native Windows; containerization optional

7.2 High-Level Architecture

```
flowchart TD
    U[Users] --> FE[React Frontend]
    FE --> API[FastAPI Backend]

    API --> AUTH[Authentication and RBAC]
    API --> DB[(PostgreSQL + pgvector)]
    API --> STORAGE[Document Storage]

    WORKER[Background Worker Process] -->|claims jobs| DB

    WORKER --> ROUTER[Document Processing Router]

    ROUTER --> PDF[PDF Processor]
    ROUTER --> OCR[OCR Processor]
```

```
ROUTER --> XLS[Excel and CSV Processor]

PDF --> TEXT[Unified Document Content]
OCR --> TEXT
XLS --> TEXT

TEXT --> LLM[LLM Extraction Service]
LLM --> SCHEMA[Structured JSON Validation]
SCHEMA --> FIN[Financial Validation]

FIN --> DB

WORKER --> CHUNK[Chunking and Embedding Job]
CHUNK --> EMB[Embedding Provider Interface]
CHUNK --> DB

DB --> REC[Reconciliation Service]
REC --> DB

DB --> SEARCH[Search Service: Keyword, Semantic, Hybrid]
SEARCH --> FE

FE --> REVIEW[Human Review Interface]
```

7.3 Architectural Principles

Separation of concerns

Each major business capability must have its own service boundary.

Suggested backend service modules:

- AuthenticationService.
- AuthorizationService.
- DocumentService.
- StorageService.

- ProcessingOrchestrator.
- PDFProcessingService.
- OCRService.
- SpreadsheetProcessingService.
- AIExtractionService.
- FinancialValidationService.
- DuplicateDetectionService.
- ReconciliationService.
- ReviewService.
- EmbeddingService.
- SearchService.
- AuditService.

These are logical service boundaries, not instructions to deploy separate microservices.

Modular monolith

Implement the MVP as a modular monolith.

Do not introduce microservices for this four-day project.

Each module should have clear responsibilities and communicate through defined interfaces.

Future modules can be extracted into independent services if scale or operational requirements justify it.

Background processing

Use background workers for expensive operations.

The API should handle requests and orchestration, while workers perform document processing and embedding generation.

Database independence

Do not mix database queries, financial rules, and API response formatting into the same business functions.

Provider independence

Wrap LLM, OCR, and embedding integrations behind interfaces so providers can be replaced without rewriting the business workflow.

Configuration-driven behavior

Keep the following configurable:

- File-size limits.
- OCR language settings.
- LLM provider/model.
- Embedding provider/model.
- Chunk size and overlap.
- Semantic similarity threshold and default search mode.
- Validation tolerances.
- Confidence thresholds.
- Job retry limits.
- Supported file types.
- Worker poll interval, maximum job attempts, retry backoff, and stuck-job timeout.
- Tesseract executable path (`TESSERACT_CMD`).
- Database connection string (`DATABASE_URL`).

8. Processing Workflow

8.1 Complete Document Processing Lifecycle

```
flowchart TD
    A[User Uploads Documents] --> B[File Validation]
    B -->|Invalid| X[Reject Upload]
    B -->|Valid| C[Create Document Record]
    C --> D[Create Background Job]
    D --> E[Document Processing Router]

    E --> F[PDF / OCR / Spreadsheet Parsing]
    F -->|Failure| Y[Processing Failed]
```

```
    F -->|Success| G[Unified Document Content]

    G --> H[LLM Extraction]
    H --> I[Schema Validation]
    I -->|Invalid Output| J[Retry or Fail]
    I -->|Valid Output| K[Save Extracted Data]

    K --> L[Financial Validation]
    L --> M[Duplicate Detection]
    M --> N[Determine Review Status]

    N -->|No Issues| O[Auto Approved / No Review Required]
    N -->|Issues Detected| P[Human Review Queue]

    P --> Q[Correction]
    Q --> R[Manager Approval]
    R --> S[Verified Record]

    N --> EMB[Create Embedding Job: Chunk and Embed Page Text]
    EMB --> T

    O --> T[Searchable Document Repository]
    S --> T
```

An embedding failure does not fail document processing. The document remains searchable by keyword and filters.

8.2 Cross-Document Reconciliation Lifecycle

```
flowchart TD
    A[User Selects Documents] --> B[Validate Document Types]
    B --> C[Load Extracted Structured Data]
    C --> D[Apply Matching Rules]
    D --> E[Compare Corresponding Fields]
    E --> F[Generate Reconciliation Results]
    F --> G[Persist Reconciliation Report]
    G --> H[Display Results in UI]
```

```
    H --> I[Human Investigation if Needed]
```

Reconciliation must be independent of the initial document-processing lifecycle.

A document can be processed successfully and reconciled later against another document.

9. Non-Functional Requirements

9.1 Performance

- Upload operations should return without waiting for document processing.
- OCR, LLM, and embedding operations must execute asynchronously.
- Document lists must support pagination.
- Database queries must use appropriate indexes, including the vector and full-text indexes.
- Processing status should be retrievable without loading extracted document content.
- Large document processing should use controlled page batches rather than unbounded memory consumption.

Do not claim a specific production throughput without performance testing.

9.2 Scalability

The architecture must support future scaling through:

- Independent worker processes.
- Configurable queue concurrency.
- External object storage.
- Database indexing and query optimization.
- Modular processing pipelines.
- Independent reconciliation execution.
- Search service abstraction.

The MVP does not need distributed deployment or horizontal autoscaling.

9.3 Reliability

- Processing failures must be recorded.
- Retryable failures should support retries.
- Jobs should be idempotent where practical.
- Original documents must remain available after processing failures.
- Extraction and validation results must be persisted.
- Reconciliation results must be independently stored.
- Embedding failures must not invalidate completed extraction results.

9.4 Security

- Use authenticated API access.
- Enforce backend RBAC.
- Hash passwords securely.
- Validate uploaded file types and sizes.
- Prevent path traversal and unsafe file handling.
- Store uploaded files outside publicly accessible application directories.
- Use parameterized database queries or ORM protections.
- Keep API credentials in environment variables.
- Restrict document download access according to user permissions.
- Apply the same document-visibility rules to search results, including semantic and hybrid search.
- Maintain audit logs for sensitive operations.

Treat uploaded documents as untrusted input. Extracted text must not be allowed to override system prompts or application instructions.

Document text is sent to external providers for extraction (LLM) and for embedding generation. The selected providers and their data-handling terms must be documented in the README.

9.5 Maintainability

- Use modular Python packages.

- Use TypeScript interfaces for frontend API contracts.
- Maintain database migrations.
- Keep business logic out of route handlers.
- Keep AI prompts in version-controlled files or a dedicated prompt module.
- Maintain structured application logging.
- Document important configuration options.

10. Testing and Evaluation

Objective: Verify that DocuLens works reliably across supported document formats and produces measurable extraction, validation, and retrieval results.

10.1 Unit Testing

Test:

- PDF text extraction.
- OCR processing.
- Excel parsing.
- CSV parsing.
- JSON schema validation.
- Invoice field normalization.
- Financial calculation validation.
- Duplicate detection.
- Reconciliation matching.
- RBAC permissions.
- Text chunking.
- Hybrid rank fusion.

10.2 Integration Testing

Test:

- Document upload and persistence.
- Upload-to-background-job creation.

- Worker execution.
- Extraction-to-validation workflow.
- Validation-to-review workflow.
- Manual correction and approval.
- PDF-to-PDF reconciliation.
- PDF-to-Excel reconciliation.
- Failed-job reprocessing.
- Embedding job creation, chunk persistence, and semantic search.

10.3 API Testing

Test:

- Authentication.
- Authorization.
- Upload endpoints.
- Processing status.
- Extraction retrieval.
- Validation retrieval.
- Search (all three modes and filters).
- Search authorization (users cannot retrieve documents they cannot view).
- Reconciliation.
- Manual corrections.
- Reprocessing.

10.4 Failure Scenarios

Test:

- Corrupted PDF.
- Empty document.
- Poor-quality scanned image.
- Unsupported format.
- Missing invoice number.
- Invalid date.
- Malformed LLM response.
- LLM API timeout.

- Embedding provider timeout or failure.
- OCR failure.
- Missing spreadsheet headers.
- Multiple Excel sheets.
- Duplicate invoice.
- Financial mismatch.
- Unauthorized API access.

10.5 Evaluation Dataset

Create a small labeled dataset containing representative examples:

- Digital PDFs.
- Scanned PDFs.
- Image invoices.
- Excel files.
- CSV files.
- Multi-page documents.
- Documents containing financial inconsistencies.

Record expected values manually as ground truth.

Create a labeled retrieval query set from the same documents. Each query lists the document IDs that should be returned. Include exact-match queries (invoice number, supplier), semantic queries (conceptual descriptions of document content), and filter-combined queries.

10.6 Evaluation Metrics

Metric	Measurement
Field extraction accuracy	Correct field values / evaluated field values
Exact invoice accuracy	Fully correct required fields / evaluated invoices
Processing time	Upload-to-processing-completion duration

Success rate	Successfully processed documents / submitted documents
Failure rate	Failed documents / submitted documents
Validation detection rate	Correctly flagged known issues / known issues
False positive rate	Incorrectly flagged valid records / evaluated valid records
Manual review rate	Documents requiring review / successfully processed documents
Straight-through processing rate	Auto-approved documents / successfully processed documents
Retrieval precision@k	Relevant documents in top k / k, per search mode
Retrieval recall@k	Relevant documents in top k / all relevant documents, per search mode

Report retrieval metrics separately for `keyword`, `semantic`, and `hybrid` modes. Use the results to set the default search mode and the semantic similarity threshold.

Use actual test results.

Do not fabricate accuracy percentages, processing speeds, or scalability claims.

11. UI/UX Expectations

The product should prioritize operational clarity over visual complexity.

Dashboard

A simple operational dashboard containing:

- Total documents.
- Processing.

- Completed.
- Failed.
- Needs review.
- Recent activity.

Document management

Users should be able to quickly:

- Upload documents.
- Find documents.
- Monitor processing.
- Open document details.
- Inspect extracted data.

Review experience

Prioritize side-by-side document inspection and extracted information.

Validation errors must be visually distinguishable.

Reconciliation experience

Display source and target values side by side.

Use clear match, mismatch, and not-comparable indicators.

Search experience

Provide a single search box with a mode selector and filters.

Show the matched snippet and page reference for each result so users can see why a document matched.

Responsive design

The application should be usable on desktop and tablet screens.

Do not spend excessive implementation time on complex animations or advanced dashboard visualizations.

12. Future Extensibility Roadmap

The following roadmap defines how the architecture should evolve after the MVP.

These features are not part of the current four-day implementation unless explicitly identified as MVP scope.

Phase 2 — Advanced Document Intelligence

1. Natural-language query parsing into structured filters.
2. Reranking and conversational retrieval.
3. Row-level spreadsheet indexing for search.
4. Advanced PDF-to-PDF reconciliation.
5. Advanced PDF-to-Excel reconciliation.
6. Automatic document relationship detection.
7. Intelligent spreadsheet column mapping.
8. More advanced line-item matching.

Required architectural foundation now:

- Persist extracted text and page-level information.
- Store chunks with the embedding model that produced them.
- Keep extraction and reconciliation independent.
- Use structured reconciliation results.
- Maintain document identifiers and relationships.

Phase 3 — Enterprise Access and Multi-Tenancy

1. Organization management.
2. Organization-level document isolation.
3. Custom roles.
4. Granular permissions.
5. Organization-specific validation policies.
6. Advanced approval workflows.

Required architectural foundation now:

- Centralized authorization.
- Explicit user ownership and audit history.
- Consistent database relationships.
- No authorization logic embedded exclusively in frontend components.

Phase 4 — Processing Scalability

1. Horizontally scalable workers.
2. Distributed task processing.
3. Advanced retry and dead-letter handling.
4. Processing prioritization.
5. Batch orchestration.
6. Worker monitoring and observability.

Required architectural foundation now:

- Database-backed asynchronous jobs with a single `run_job(job_id)` entry point.
- Persistent job states.
- Independent worker execution.
- Idempotent processing operations.
- Configurable retries and timeouts.
- Migration to a dedicated message broker (Redis with Celery or equivalent)

Phase 5 — Analytics and Business Intelligence

1. Processing performance analytics.
2. Supplier-level financial insights.
3. Document discrepancy trends.
4. Extraction accuracy monitoring.
5. Human review performance.
6. Operational reporting.

Required architectural foundation now:

- Processing history.
- Validation history.

- Reconciliation history.
- Correction history.
- Consistent timestamps and event records.

13. Four-Day Implementation Plan

The project must prioritize a working vertical slice while covering all mandatory requirements.

Day 0 — Environment Spike (about one hour)

- Install Tesseract and OCR one sample image.
- Connect to the hosted database; enable `vector` and `pg_trgm`.
- Run an empty Alembic migration.
- Start the API and the worker loop; process one dummy job end to end.

Day 1 — Foundation and Document Processing

Backend

- FastAPI project structure.
- PostgreSQL connection with pgvector enabled.
- Database models and migrations (`document_chunks` is created on Day 2 after the embedding model is selected).
- Authentication and RBAC foundation.
- User management APIs.
- Document upload APIs.
- Local document storage.
- Database-backed job queue and worker process.
- PDF, image, Excel, and CSV processing.

Frontend

- React project setup (Vite, Tailwind, shadcn/ui).
- Login page.

- Application layout.
- Document upload interface.
- Basic document list.

End-of-day deliverable:

Users can authenticate, upload multiple supported document formats, and create asynchronous processing jobs.

Day 2 — AI Extraction, Financial Validation and Embedding Pipeline

Backend

- Select the embedding provider and model; create the `document_chunks` migration.
- LLM integration.
- Structured extraction schema.
- Invoice extraction prompts.
- JSON validation.
- Field normalization.
- Financial validation.
- Duplicate detection.
- Processing status updates.
- Text chunking and embedding job (EMBEDDING job type, provider interface).

Frontend

- Document details page.
- Extracted information display.
- Validation results display.
- Processing status monitoring.

End-of-day deliverable:

Uploaded documents can be processed into structured invoice information, validated automatically, and indexed for semantic search.

Day 3 — Human Review, Reconciliation and Search

Backend

- Review queue.
- Manual correction APIs.
- Approval workflow.
- Audit history.
- PDF-to-PDF reconciliation.
- PDF-to-Excel reconciliation.
- Keyword, semantic, and hybrid search with filters and authorization.
- Failed-job reprocessing.

Frontend

- Review interface.
- Correction and approval controls.
- PDF-to-PDF comparison.
- PDF-to-Excel comparison.
- Search and filtering (mode selector, snippets).

End-of-day deliverable:

The primary business workflows are operational.

Day 4 — Integration, Testing and Delivery

Backend

- API integration testing.
- Failure handling.
- Authorization verification (including search).
- Retrieval evaluation (labeled query set, per-mode metrics, threshold and default-mode selection).
- Performance improvements.
- Database query optimization.

Frontend

- Basic user management interface.
- End-to-end integration.
- Error handling.
- Loading states.
- Responsive layout.
- UI consistency.

Documentation and delivery

- README.
- Architecture diagram.
- Database schema.
- API documentation.
- Test report.
- Extraction and retrieval evaluation report.
- Walkthrough recording.
- AI prompts and AI-assisted development disclosure.

End-of-day deliverable:

A working, documented, testable application with all mandatory MVP capabilities.

Important: This is an aggressive schedule. Build the vertical slice early, avoid unnecessary UI polish, and do not leave testing and documentation until the final hours.

14. Definition of Done and Acceptance Criteria

The project should not be considered complete merely because the application starts successfully.

The following acceptance criteria must be satisfied.

A. Authentication and RBAC

- Users can log in.
- Four predefined roles exist.
- Backend enforces role permissions.
- Unauthorized operations are rejected.
- Administrators can manage users.

B. Document Processing

- PDF upload works.
- JPG/PNG/TIFF upload works.
- XLSX/XLS/CSV upload works.
- Multi-page PDFs are supported.
- Scanned documents use OCR.
- Multiple Excel sheets are supported.
- Corrupted documents fail gracefully.

C. AI Extraction

- Invoice number is extracted.
- Invoice date is extracted.
- Supplier name is extracted.
- Customer name is extracted.
- Subtotal is extracted.
- Tax is extracted.
- Total is extracted.
- Currency is extracted.
- Line items are extracted.
- Missing fields are represented without hallucinated values.
- Output follows a validated JSON schema.

D. Financial Validation

- Required fields are checked.

- Numeric values are validated.
- Financial calculations are checked.
- Duplicate candidates are identified.
- Validation issues are persisted.
- Documents requiring review are flagged.

E. Human Review

- Review queue works.
- Original document is accessible.
- Extracted values are editable by authorized users.
- Corrections are persisted.
- Original extracted values are preserved.
- Finance Manager can approve or reject records.
- Audit history is maintained.

F. Reconciliation

- PDF-to-PDF comparison works.
- PDF-to-Excel comparison works.
- Excel sheet selection works.
- Spreadsheet column mapping works.
- Matching and mismatching fields are displayed.
- Reconciliation results are persisted.

G. Search and Processing

- Keyword search works.
- Document filtering works.
- Page text is chunked and embedded after processing.
- Semantic search returns relevant documents with matched snippets.
- Hybrid search works and falls back to keyword results if embedding fails.
- Search results respect user authorization.
- Processing status is retrievable.
- Failed jobs can be retried by authorized users.
- Background processing does not block upload requests.

H. Quality and Delivery

- Automated tests are available.
- Actual extraction and retrieval metrics are reported.
- Failure scenarios are tested.
- README contains setup instructions.
- Architecture diagram is provided.
- Database schema is documented.
- Walkthrough video is recorded.
- AI prompts and AI-assisted development are disclosed.

15. Engineering Implementation Guidelines

This section is specifically intended for the Technical Project Manager and AI-assisted engineering tools such as Antigravity and Claude Code.

15.1 General Engineering Instructions

1. Read and understand the complete PRD before implementation.
2. Do not implement features outside the defined scope without explicit approval.
3. Use FastAPI for the backend and React + TypeScript for the frontend.
4. Use PostgreSQL (with pgvector) as the primary database for both relational data and vector search.
5. Use database-backed background workers for asynchronous processing.
6. Follow a modular monolith architecture.
7. Keep business logic separate from API routes.
8. Use database migrations.
9. Implement proper exception handling.
10. Validate all external AI responses.
11. Do not trust LLM output without schema and business validation.
12. Never fabricate extracted financial information.
13. Preserve original documents and extraction history.
14. Implement authorization at the backend.

15. Keep all secrets and credentials outside source code.
16. Maintain API contracts between frontend and backend.
17. Write tests alongside critical business logic.
18. Avoid unnecessary abstractions and premature microservices.

15.2 AI Engineering Guidelines

The AI engineer must understand that DocuLens is not primarily a conversational AI application.

The LLM is a specialized component within a broader deterministic document-processing system.

The LLM should be responsible for:

- Understanding extracted document content.
- Identifying relevant invoice information.
- Mapping unstructured content to structured fields.
- Handling variations in invoice layouts.
- Returning structured extraction results.
- Indicating uncertainty.

The LLM must not be the authority for:

- Financial calculation validation.
- Duplicate confirmation.
- Final reconciliation decisions.
- User authorization.
- Processing status management.
- Approval decisions.

These responsibilities belong to deterministic application services.

Embeddings are used only for retrieval ranking. They must not be used to make financial, validation, or approval decisions.

15.3 AI Prompt Management

Keep prompts version-controlled.

Each prompt should specify:

- Extraction objective.
- Supported fields.
- Expected JSON schema.
- Missing-value behavior.
- No-hallucination instruction.
- Field normalization requirements.
- Output restrictions.

Avoid scattering prompt strings across unrelated backend modules.

15.4 Development Workflow

Recommended implementation sequence:

1. Establish project structure.
2. Define database schema and API contracts.
3. Implement the document upload vertical slice.
4. Implement document processing.
5. Integrate AI extraction.
6. Implement deterministic validation.
7. Implement chunking and the embedding pipeline.
8. Implement review, reconciliation, and search.
9. Integrate frontend.
10. Run tests and retrieval evaluation.
11. Complete documentation.

The frontend and backend can be developed in parallel after agreeing on API contracts and shared data structures.

15.5 AI-Assisted Development Instructions

When using Antigravity or Claude Code:

- Provide the complete PRD as project context.
- Ask the agent to inspect the existing repository before modifying files.
- Ask it to explain the implementation plan before major structural changes.

- Implement one module at a time.
- Require runnable code rather than pseudocode.
- Ask for tests for critical business logic.
- Verify database migrations.
- Verify API authorization.
- Review generated prompts and LLM response schemas.
- Do not accept fabricated test results.
- Do not allow the agent to silently change architectural decisions.

16. Key Product Decisions and Assumptions

The following decisions establish the MVP implementation baseline.

Area	Decision
Backend	FastAPI
Frontend	React + TypeScript, Tailwind CSS + shadcn/ui
Architecture	Modular monolith
Database	PostgreSQL + pgvector, Hosted PostgreSQL (Supabase or Neon) via DATABASE_URL
Background processing	PostgreSQL-backed job queue + Python worker process
OCR	Tesseract or PaddleOCR
PDF extraction	PyMuPDF
Spreadsheet parsing	pandas + openpyxl
AI extraction	External LLM API
Search	PostgreSQL keyword search + structured filters

Semantic search	pgvector in PostgreSQL; hybrid via Reciprocal Rank Fusion
Embedding provider	Selected during implementation, behind a provider interface
Natural-language query parsing	Deferred
Reconciliation	Simple field-level comparison
Authentication	JWT-based
Authorization	Four predefined roles
Multi-tenancy	Deferred
File storage	Local storage abstraction
Vector database	Not used; pgvector in PostgreSQL
Distributed processing	Deferred
Analytics	Basic operational statistics
Local development	Native Windows; containerization optional

Additional assumptions requiring confirmation

1. The MVP operates within a single organization.
2. The MVP primarily processes invoice-related financial documents, even though receipts and other supporting documents are accepted as inputs.
3. PDF-to-PDF reconciliation initially compares two explicitly selected documents.
4. PDF-to-Excel reconciliation initially compares one PDF against a selected Excel worksheet.
5. Excel column mapping is explicitly configured by the user.
6. The MVP does not automatically discover relationships between unrelated documents.
7. A Finance Manager is responsible for final approval of corrected records.

8. Documents that pass automated checks do not require operational approval.
9. The system stores original documents and their extracted information.
10. Performance and accuracy targets will be established from actual evaluation results.
11. The visibility rule for "Authorized" documents (Finance Executive and Accounts/Audit roles) must be defined once and reused by the document list, document access, and search.
12. Semantic search covers text extracted from PDFs and images. Spreadsheets are searchable by filename and metadata only.
13. The embedding provider and model are selected during implementation and recorded in the README.
14. The hosted PostgreSQL service supports the pgvector and pg_trgm extensions and allows direct or session-mode connections.
15. Reviewers provision their own PostgreSQL instance and run migrations; database credentials are never shared in the repository.

17. Final Product Direction

DocuLens should be developed as a modular financial document intelligence platform with a working MVP and a clear evolution path toward enterprise-grade capabilities.

The initial implementation must deliver:

- Multi-format document ingestion.
- Automated document extraction.
- AI-powered structured invoice information extraction.
- Deterministic financial validation.
- Duplicate detection.
- Exception-based human review.
- PDF-to-PDF reconciliation.
- PDF-to-Excel reconciliation.
- Four-role RBAC.
- Asynchronous document processing.
- Keyword and semantic document search with structured filters.
- Processing and audit history.

At the same time, the implementation must establish the architectural foundation for:

- Natural-language and conversational retrieval.
- Intelligent document reconciliation.
- Enterprise authorization.
- Multi-tenancy.
- Distributed document processing.
- Advanced business analytics.

The objective is not to build every future capability now. The objective is to build today's capabilities correctly, with clean interfaces and data structures that allow tomorrow's capabilities to be introduced without rewriting the core system.

Final engineering principle: Build for extensibility, implement for the current requirements, and optimize for a reliable working product within the four-day deadline.

Technology stack

Layer	Technology	Role
Frontend	Vite, React, TypeScript	SPA with role-aware routing
UI	Tailwind CSS, shadcn/ui, TanStack Query, react-hook-form + zod	Components, server-state caching, form validation
API contract	FastAPI OpenAPI, `openapi-typescript`	Generated TS types, no hand-written drift
Backend API	Python, FastAPI, Pydantic v2, uvicorn	REST, validation, RBAC dependencies
Data access	SQLAlchemy 2.0 (sync), psycopg3, Alembic	Repositories and migrations
Database	PostgreSQL + pgvector + pg_trgm (hosted: Supabase or Neon)	Relational data, vectors, keyword search, job queue
Job queue	`processing_jobs` table, `FOR UPDATE SKIP LOCKED`, `python -m app.worker`	Async work with retries, leases, backoff
PDF text	PyMuPDF	Text layer extraction and page rendering for OCR

OCR	Tesseract via pytesseract (`TESSERACT_CMD`)	Scanned PDFs and images
Spreadsheets	pandas, openpyxl, xlrd	XLSX, XLS, CSV with sheet and row traceability
Upload validation	Magic-byte checks (no native dependency)	Real file type, not just extension
LLM	Provider interface, tenacity retries, Pydantic schema, `Decimal` amounts	Extraction only, never final validation
Embeddings	Provider interface, model pinned in config	Chunk vectors for semantic search
Search	`tsvector` + pg_trgm + pgvector, RRF for hybrid	Keyword, semantic, hybrid, filtered before ranking
Auth	PyJWT, bcrypt or argon2	JWT sessions, four-role RBAC
Config	pydantic-settings, `.env`	Providers, tolerances, thresholds, paths
Testing	pytest, httpx, fake LLM and embedding providers	Deterministic tests
Evaluation	Standalone CLI over the labeled dataset	Measured extraction and retrieval metrics
Run	Native Windows: uvicorn, worker, Vite dev server	Containerization optional later